Bounded Context Is Not Bounded Information: Structured State Memory Trades Capability for Prompt Size in Long-Horizon Coding Agents

[**NOTE:** authors and affiliation pending owner decision]

Draft September 20, 2026

Abstract

Structured state memory replaces an agent's conversation history with a compact, model-maintained state block. On synthetic long-horizon benchmarks this bounds prompt growth at near-constant size and reports no loss of task quality. We test whether that result transfers to real coding work, on a closed pre-registered design: 87 Terminal-Bench 2.0 tasks × 4 conditions × 3 independent trials = 1044 runs, every cell filled, verdicts taken only from each task's own verifier.

It does not transfer in this regime. The method as published solves 0.349 of tasks against the transcript control's 0.613; paired on (task, trial), it loses 82 tasks and wins 13 ($p = 2.1 \times 10^{-13}$). The loss is *larger* on long-horizon tasks (54 vs. 8, $p = 1.7 \times 10^{-9}$) than on short ones, the opposite of what a memory mechanism is meant to buy. Restoring six verbatim exchanges beside the state block leaves no detectable deficit (29 vs. 23 discordant tasks, $p = 0.49$), which locates the damage in the discarded history rather than in the state block itself. We report this as a failure to detect a difference, not as demonstrated equivalence.

The runs show the mechanism. Bounding the *prompt* does not bound *information acquisition*: agents re-acquire discarded context through their tools, repeating byte-identical inspection commands at 58 % of all commands in failing state-only runs against 20 % for the control, and following pointers into files 12 times more often, so the tool loop spends in turns what the prompt saved in context. The economics follow: at equal outcome the state-only arm carries 0.47× the control's peak prompt yet costs 2.57× as much, because state maintenance is billed at the output tariff (4.55×) and a state block rewritten each turn destroys the prefix cache the control enjoys (352.2M cached tokens against 6.2M).

The tested regime is one model family on one benchmark, and it is a regime in which the control never approaches its context limit, so these results bound what the mechanism buys when the window is not the binding constraint rather than when it is.

Our contributions are a falsification of the transfer claim on real coding tasks, an identification of the evaluation property that concealed it (fixing the number of model calls per episode makes compensatory re-acquisition impossible by construction), a cache-aware accounting method under which token reduction and cost reduction are shown to be different quantities, and a released dataset of 1044 fully instrumented agent trajectories.

Reading the source tags. Claims about our own system carry ^{live} for a measurement on a real run, ^{code} for a property of the code, and ^{orig} for a statement of the original papers. Anything without a tag is argument, not evidence.

1 Introduction

A coding agent that works for hours accumulates a transcript that grows without bound. Every observation, every tool result and every intermediate thought is re-sent on the next model call,

so the prompt grows with the task and the cost of a turn grows with everything that preceded it. Structured state memory is the most direct answer to this: discard the transcript, and have the model maintain a compact structured state that is supposed to be a sufficient statistic for what remains to be done. SKILL.STATE [Badhe et al., 2026] reports that this holds prompt size near-constant from 10 to 200 environment events while quality does not degrade[orig].

That result is measured on a synthetic generator. This paper asks whether it survives contact with real software tasks, where the agent decides for itself what to inspect and when it is finished.

The claim we test. Not "is structured state useful" in general, but the specific transfer claim that follows from the published evaluation: that replacing the transcript with a maintained state block preserves task capability while bounding context. We test it on Terminal-Bench 2.0, with a design fixed in writing before any run (§4) and verdicts taken only from each task's own verifier.

The result. It does not transfer. On 1044 runs the published configuration solves 0.349 of tasks against 0.613 for the transcript control, losing 82 paired tasks and winning 13 ($p = 2.1 \times 10^{-13}$). This is not an artefact of a hard corpus or a weak model: the same model on the same tasks with its history intact solves them.

Two further observations turn this from a negative result into an explanation. First, the damage *grows* with task horizon (§7), exactly inverting the argument for the method, whose benefit is supposed to accumulate on long tasks. Second, restoring a mere six verbatim exchanges alongside the state block leaves a deficit we can no longer detect ($p = 0.49$; absence of evidence, not evidence of equivalence), which places the loss in the discarded history rather than in the state abstraction.

Why the original evaluation could not see this. In the synthetic setting the number of model calls per episode is fixed by construction: the environment emits T events and every runtime answers each one exactly once[orig]. An agent that has forgotten something therefore *cannot* pay to recover it: there is no mechanism in the design by which re-acquisition could appear as a cost. In an agentic setting the step count is an outcome, not a control, and that is precisely where the price appears (§6).

The mechanism, stated as the paper's thesis.

> *Bounded conversational context does not imply bounded effective information acquisition, because coding agents compensate by re-reading the environment.*

The state block bounds what the agent is *told*; it does not bound what the agent *needs*. The difference is paid through the tool loop, which no prompt-size metric observes. We measure it directly: in failing state-only runs 58 % of all commands are repeats of an inspection already performed, against 20 % for the control[live] (§8).

Economics follows from the same mechanism. Because state maintenance is emitted as generated text, it is billed at the output tariff; because the block is rewritten every turn, it destroys the prefix cache from which an append-only transcript is almost entirely served. At equal outcome the state-only arm carries 0.47× the control's peak prompt and costs 2.57× as much[live]. A token-count reduction is not a cost reduction, and on this corpus it is not even a cost reduction in the direction the token count suggests.

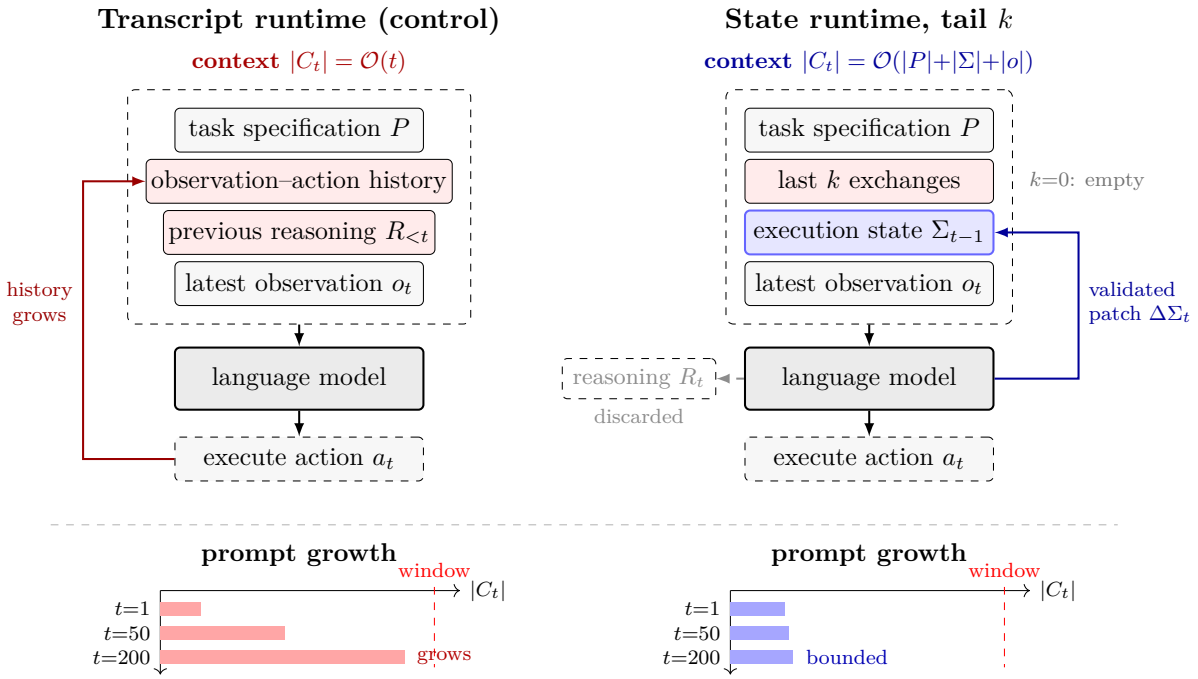


Figure 1: The two context policies of Algorithms 1 and 2. **Left:** the transcript runtime appends every exchange, so the prompt grows with the episode (Eq. 1) and eventually meets the context window. **Right:** the state runtime carries a validated state block and, at k=0, no history at all (Eq. 3); its prompt is bounded independently of t. Both discard intermediate reasoning. The bounded prompt is the published benefit. What the diagram cannot show is the quantity this paper measures: bounding $|C_t|$ does not bound the information the task requires, so the agent re-acquires it through tool calls, inflating the realised horizon (Eq. 6) at no cost visible to any prompt-size metric.

Contributions.

1. A falsification, on a closed pre-registered design of 1044 runs, of the claim that structured state memory preserves capability on real coding tasks (§7).
2. An identification of the evaluation property that concealed it: fixing the per-episode model-call count makes compensatory re-acquisition unmeasurable by construction (§6).
3. A mechanism, measured at the level of individual tool calls, connecting discarded history to redundant environment inspection (§8).
4. A cache-aware cost methodology under which token reduction and monetary reduction are shown to be distinct, and under which no configuration of structured state is cheaper than a transcript at equal outcome (§9).
5. A released dataset of 1044 fully instrumented trajectories with per-run provenance, and the deterministic analysis that regenerates every number in this paper (§A).

What we do not claim. We do not claim that structured state is a bad idea. We claim that its published evaluation measures a regime in which its central failure mode cannot occur, and that on real coding tasks the mechanism trades capability for prompt size at a rate the original metric cannot express. Section 11 states the boundaries: one model family, one corpus, and a composition study still in progress.

2 Background

2.1 Structured state memory

SKILL.STATE [Badhe et al., 2026] maintains a persistent structured state Σ_t and assembles each prompt as

$$\text{prompt}_t \;=\; [\,\text{task spec}\,] \;+\; [\,\Sigma_t\,] \;+\; [\,o_t\,],$$

where o_t is the latest observation and the interaction history is discarded. The model emits reasoning, a state patch and an action in a single generation, $(R_t, \Delta\Sigma_t, a_t)$, as one JSON object with exactly the two keys `state_patch` and `action`[orig]. The runtime validates each patch against a schema and rejects invalid ones with a rollback-retry cycle. Schemas are authored once per domain; across 100 InterCode CTF instances a single five-field schema is reused[orig].

The reported evidence is a horizon sweep on a procedural generator: prompt size stays near 1800 characters from T=10 to T=200 while baselines grow to 48,000, and quality is preserved or improved[orig]. The paper's own error taxonomy for open-weight models attributes 68 % of failures to premature state overwrite or deletion[orig], a figure we return to in §10, because it is an argument for moving part of the state out of the model's hands.

2.2 Deterministic execution state

LEDGER [Wang et al., 2026] occupies the opposite design corner: a runtime-maintained record derived mechanically from completed interactions: observation records, per-file and global modification counters, normalised command records with their outcomes. It adds no model calls and does not modify the agent. Its two mechanisms separate cleanly in its own ablation: *inform* (rendering the record at the end of the input, where it cannot disturb an append-only prefix) yields most of the efficiency gain, while *govern* (intercepting a proposed command and reusing an earlier result) yields most of the resolution gain[orig].

The two systems are complementary rather than competing. SKILL.STATE keeps only semantic state and keeps it in the model; LEDGER keeps only objective state and keeps it in the runtime. Our composition study (§10) varies precisely this boundary, and we deliberately exclude *govern* from it: govern changes what the agent *does*, so including it would mix a large effect into the measurement of a smaller one.

2.3 The assumptions under test

Stated as propositions that can be checked rather than as criticisms:

A1 (schema availability)
An adequate domain schema exists before execution. The original gives no schema-design method and reports a verbatim field list for only one of its four domains[orig].

A2 (positive from turn one)
Maintaining state is at worst neutral from the first turn. The shortest horizon reported is T=10, already longer than a large share of real coding tasks.

A3 (sufficient statistic)
Nothing discarded is ever needed again. The original names this as a limitation[orig] and does not measure it; it is the assumption this paper falsifies.

M (measurement currency)
Cumulative token count is a valid proxy for cost. A full-text search of the original returns zero occurrences of cache, billing or monetary terminology[orig].

A1 and A3 are properties of the method. M is a property of how the method was *evaluated*, and it is the one that makes the other two hard to see: a metric that counts logical tokens

cannot represent either the cache discount that rewards an append-only prefix or the tool calls spent re-acquiring what the prompt no longer carries.

2.4 The design axis this paper uses

Rather than comparing named architectures, we hold the mechanism fixed and vary one integer: the number k of most recent verbatim exchanges retained beside the state block.

$$\text{prompt}_t \;=\; [\,\text{task spec}\,] \;+\; [\,\text{last } k \text{ exchanges}\,] \;+\; [\,\Sigma_t\,]$$

At k=0 this is exactly SKILL.STATE as published. As k grows the configuration interpolates toward a transcript agent. The control is not a point on this axis (it carries no state block at all) and is always reported separately.

This buys three things a three-arm comparison cannot. It removes the arbitrary constant that an intermediate "hybrid" arm would smuggle in; it turns the question from *which architecture* into *where on the axis the optimum lies*; and it makes the shape of the dependence observable, which is what ultimately localises the damage (§7).

3 Formal model

This section fixes notation, states the two runtimes as algorithms, and derives the accounting identity that the rest of the paper measures against. The asymptotic argument is the original's[orig]; our contribution here is to show which of its terms the standard evaluation holds constant, and what the identity becomes once that term is released.

3.1 Notation

An agent executes a task over discrete turns $t = 1, 2, \ldots$ Let P be the task specification, o_t the observation returned by the environment at turn t, a_t the action the agent takes, and R_t its intermediate reasoning. Write Σ_t for the structured execution state and C_t for the context assembled for the model call at turn t. We use $|\cdot|$ for size in tokens.

The *horizon* T is the number of turns the episode actually takes. Whether T is an input or an output is the distinction on which §6 turns, and we mark it explicitly: T^{env} when the environment fixes it, T^{ag} when the agent stops of its own accord.

3.2 Two context policies

The transcript runtime appends every exchange, so its context at turn t is

$$C_t^{\text{tr}} \;=\; [\,P,\; (o_1, R_1, a_1), \ldots, (o_{t-1}, R_{t-1}, a_{t-1}),\; o_t\,], \tag{1}$$

which grows linearly in t, giving $|C_t^{\text{tr}}| = \mathcal{O}(t)$ and cumulative prompt volume

$$\sum_{t=1}^{T} |C_t^{\text{tr}}| \;=\; \mathcal{O}(T^2). \tag{2}$$

The structured-state runtime discards the history and carries only the state:

$$C_t^{\text{st}} \;=\; [\,P,\; \Sigma_{t-1},\; o_t\,], \qquad |C_t^{\text{st}}| \;=\; \mathcal{O}\big(|P| + |\Sigma| + |o|\big), \tag{3}$$

which is independent of t whenever the state is bounded, so

$$\sum_{t=1}^{T} |C_t^{\text{st}}| \;=\; \mathcal{O}(T). \tag{4}$$

Equations (2) and (4) are the published argument, and we reproduce them without dispute[orig].

3.3 The tail family that interpolates between them

We generalise both into one family indexed by an integer $k \geq 0$, the number of most recent exchanges retained verbatim beside the state:

$$C_t^{(k)} \;=\; [\, P,\; (o_{t-k}, R_{t-k}, a_{t-k}), \ldots, (o_{t-1}, R_{t-1}, a_{t-1}),\; \Sigma_{t-1},\; o_t \,]. \tag{5}$$

At $k = 0$ this is exactly (3); as $k \to t$ it recovers (1) with a redundant state block. The control carries no state block at all and is therefore not a member of the family; it is reported separately throughout.

Every condition in this paper is a value of k in (5), which is what makes the comparison a position on a curve rather than a contest between named systems.

3.4 The runtimes as algorithms

Algorithm 1 Transcript runtime (control)

Require: task specification P
1: $H \leftarrow \varnothing$ ▷ interaction history
2: **for** $t = 1, 2, \ldots$ **do**
3: receive observation o_t
4: $C_t \leftarrow [\, P,\; H,\; o_t \,]$
5: generate $(R_t, a_t) \sim \pi(C_t)$
6: execute a_t
7: $H \leftarrow H \,\|\, (o_t, R_t, a_t)$
8: **if** agent declares completion **then**
9: **break**
10: **end if**
11: **end for**

Algorithm 2 State runtime with tail k

Require: spec P; schema S; tail k
1: $\Sigma_0 \leftarrow \varnothing$; $H \leftarrow \varnothing$
2: **for** $t = 1, 2, \ldots$ **do**
3: receive observation o_t
4: $C_t \leftarrow [\, P,\; \mathrm{tail}_k(H),\; \Sigma_{t-1},\; o_t \,]$
5: generate $(R_t, \Delta\Sigma_t, a_t) \sim \pi(C_t)$
6: **if** $\mathrm{valid}_S(\Delta\Sigma_t)$ **then**
7: $\Sigma_t \leftarrow \Sigma_{t-1} \oplus \Delta\Sigma_t$
8: **else**
9: $\Sigma_t \leftarrow \Sigma_{t-1}$; return error to model
10: **end if**
11: execute a_t; discard R_t
12: $H \leftarrow H \,\|\, (o_t, R_t, a_t)$
13: **if** agent declares completion **then**
14: **break**
15: **end if**
16: **end for**

The two runtimes differ in exactly one respect: what line 4 assembles. The merge $\oplus$ applies a schema-validated patch, and an invalid patch leaves the state untouched. At k=0 the tail is empty and Algorithm 2 is SKILL.STATE as published. Both discard the reasoning R_t; only Algorithm 2 also drops the history from the context.

The termination test in both (line 8 of Algorithm 1 and line 13 of Algorithm 2) is the agent's own declaration of completion, not the expiry of an external counter. This is the only structural difference from the original's formulation, where the loop runs for a fixed T^{env}, and §6 shows it is the difference that matters.

3.5 What the standard accounting omits

Let $\mathcal{I}_t$ denote the information the agent requires at turn t to choose a correct action. It can arrive by two routes: carried in the context, or re-acquired from the environment by spending a turn on an inspection action. Write ρ for the fraction of turns spent on re-acquisition.

A transcript agent has already paid for its history (it is in the prompt), so ρ^{tr} is small. A state agent has discarded whatever Σ does not encode, so any needed fact outside Σ must be re-acquired, and $\rho^{(k)}$ grows as k shrinks. Since each re-acquisition consumes a turn, the realised horizon is inflated:

$$T^{\mathrm{ag}}(k) \;\approx\; \frac{T_0}{1 - \rho(k)}, \tag{6}$$

where T_0 is the turn count an agent with perfect recall would need.

Substituting (6) into (4) gives the quantity the original's asymptotics do not contain:

$$\sum_{t=1}^{T^{\mathrm{ag}}} |C_t^{\mathrm{st}}| \;=\; \mathcal{O}\Big(\frac{T_0}{1-\rho}\Big), \qquad \rho = \rho(k). \tag{7}$$

The published comparison evaluates (4) against (2) with $T = T^{\mathrm{env}}$ held equal across arms. That choice sets ρ outside the model: with the turn count fixed by the environment, re-acquisition cannot lengthen an episode, and (6) degenerates to $T^{\mathrm{ag}} = T^{\mathrm{env}}$ for every runtime regardless of how much was forgotten.

This is the paper's central formal point. The asymptotic advantage of (4) is real, and it is stated in a variable (T) that the evaluation holds constant while the mechanism under test (ρ) acts precisely on that variable. We measure ρ directly in §8 and T^{ag} in §7.

3.6 Cost is not token count

Let α, γ, β be the per-token prices of uncached input, cached input and output. With h_t the cached fraction of the prompt at turn t, the billed cost of an episode is

$$\mathrm{Cost} \;=\; \sum_{t=1}^{T} \Big[\, \alpha\,(1-h_t)\,|C_t| \;+\; \gamma\, h_t\,|C_t| \;+\; \beta\,|y_t| \,\Big], \tag{8}$$

where y_t is the generated output, which for a state runtime includes the patch $\Delta\Sigma_t$. On current providers $\gamma \ll \alpha < \beta$.

Two consequences follow, and both are measured in §9. A transcript context is append-only, so C_t^{tr} extends C_{t-1}^{tr} and $h_t \to 1$: the growing prompt of (2) is billed mostly at the cheap rate γ. A state block is rewritten each turn, so the prefix diverges at Σ and h_t stays low despite the small $|C_t|$. Meanwhile maintaining state moves tokens from the $|C_t|$ terms into $|y_t|$, which carries the highest price β.

Hence the identity this work is built on: minimising $\sum_t |C_t|$ and minimising (8) are different objectives, and on the corpus measured here they have opposite solutions.

4 Study design and protocol freeze

This section is placed before the results deliberately. The design was fixed in writing before the series ran, and the sequence *hypothesis → frozen method → data → interpretation* is what allows a reader to distinguish a finding from a story assembled after the fact.

4.1 What was frozen, and when

The pre-registration is a repository artefact dated before the first run of the series[code]. It fixes the question, the hypotheses with their falsification conditions, the intervention, the dependent variables, the confounders and their controls, the sample size with its power rationale, the stopping rule, the statistical tests, and the exclusion policy. Its amendment section is empty: nothing was changed after the first run started.

Three commitments in it do the real work:

1. **The verdict is external.** A run's outcome is the exit status of the task's own verifier, never the agent's report of its own success. A run without a verifier verdict does not enter the dataset at all.
2. **Tests were chosen before the data.** Paired exact McNemar for capability, exact sign test on per-pair ratios for cost, Spearman rank correlation for the horizon trend. No test was selected after seeing an effect.

3. **Exclusions were defined before the counts were seen.** The contamination rule (§12) and the duplicate-collapse rule were both written before they were applied. The latter keeps the *earliest* launch of a re-run cell, so a repeat cannot promote itself into the sample by winning.

4.2 Hypotheses and how each could fail

H1 (cost).
The median cost ratio of state-only to control exceeds 1.0. *Refuted if* the ratio is ≤ 1.0, the sign test misses significance, or the direction is inconsistent across the three trials.

H2 (horizon).
The cost ratio falls as horizon grows, crossing 1.0: the overhead amortises on long tasks. *Refuted if* the correlation is absent, positive, or never approaches 1.0 in the observed range.

H3 (capability).
Reported with its interval and treated as underpowered unless the observed discordance exceeds the pre-computed requirement of six one-sided discordant pairs.

H2 is the hypothesis this study refutes in the pre-registered manner, and we report it as such in §9 rather than quietly dropping it.

4.3 Multiple comparisons

Three hypotheses were pre-registered, and those three carry the study's confirmatory weight. Everything else this paper reports a p-value for, the per-stratum splits, the ladder steps of §10, the per-condition cost comparisons, is secondary and exploratory, and no family-wise correction is applied to it.

We state this explicitly because §B criticises the original for not doing so, and the same standard has to apply here. The consequence is concrete: our headline result survives any correction one would care to apply ($p = 2.1 \times 10^{-13}$ on the primary comparison, against a Bonferroni threshold of 0.05/3 for the confirmatory family), but several secondary results sit close enough to 0.05 that they would not. Where a secondary result is load-bearing for a claim in this paper we say so and treat it as provisional rather than established.

4.4 A pre-declared artefact trap

One trap was named in advance because it would have produced a flattering result: if the state-only arm showed a *lower* cost on long tasks merely because long tasks are where it more often exhausts its turn budget and stops early, that is censoring, not amortisation. The pre-registration therefore excludes budget-stopped runs from cost aggregation and reports their count per stratum separately. The trap was real: the stop-reason distribution in Table 4 shows budget stops concentrated in exactly that arm.

4.5 Scope

This series addresses the cost and horizon questions, and reports capability. Schema sensitivity and state composition are separate designs with their own pre-registrations; their status is given in §10. [**PENDING:** if the owner elects to publish E1 alone, this paragraph becomes the forward reference to the second paper]

5 Experimental setup

5.1 Agent, corpus and conditions

The agent is an unmodified open-source coding agent; the state mechanism is a plugin that replaces the context assembled for each model call and adds a validated patch tool[code]. Versions, plugin content hash and schema hash are frozen for the series and recorded in every run row (plugin `be0f5b73e526ba4a`, schema `7d8705aae4e124f1`)[live].

The corpus is Terminal-Bench 2.0. Of 89 tasks runnable in principle, this stand reaches 87: two fail during container setup, before the agent starts, on every attempt and in every condition[live]. The model is `zai/glm-5.3-flash` throughout.

The four conditions differ in exactly one property:

Condition	Context assembled for each model call
CONTROL	full transcript, no state block (unmodified agent)
STATE-ONLY	state block replaces the transcript (k=0)
STATE+2	state block + 2 most recent exchanges
STATE+6	state block + 6 most recent exchanges

Everything else is frozen: same model, schema, tools, turn budget, task and verifier. The realised k is read back out of each run's trace, and a run whose realised value disagrees with the requested one is rejected rather than analysed[code].

5.2 Unit, pairing and repetition

The experimental unit is one run of one task under one condition in one trial. Every comparison is paired within *(task, trial)* against the control of the *same* trial, never against a pooled control: single runs of an identical cell have been measured at 15, 51 and 65 model calls[live], so cross-trial pooling would compare a condition against a different sample of the same noise. Three independent trials give 1044 runs in total.

5.3 Verdicts

The verdict is the exit status of the task's own verifier. This is not a formality: in an earlier phase of this project a model's self-report and a broken verifier container between them produced a full series of rows that looked like data and were not (§12).

Runs are additionally classified by how they ended (output_ceiling, solved, turn_budget, unsolved), and the distribution is reported in Table 4, because scoring a budget stop or an output-ceiling stop as an ordinary failure attributes a harness property to the method.

5.4 Cost accounting

Per-call usage (input, cached-read and output tokens) is taken from the session journal. Cost is derived from those measured tokens at one fixed published price table, not read from the provider, for two measured reasons: the account's two wallets bill identical token counts exactly a factor of two apart[live], and the custom-endpoint path reports zero cost while returning real tokens[live]. The provider's own figure is retained in the raw records for audit.

Cached-read tokens are tracked separately from input tokens throughout, because the central methodological claim is that they are different currencies. That the provider's prefix cache is real and visible in the usage fields was verified before the series: a repeated call moved from 0 to 3008 cached tokens of 3019[live].

5.5 Gates run before any measurement

A stack that has changed is smoke-tested before it is trusted[code]:

1. an oracle solution passes the verifier, establishing that the verifier can recognise a correct answer at all;
2. every arm demonstrably loads the plugin and realises its declared k, checked against the assembled prompt rather than the plugin's own log; a silent import failure otherwise turns a treatment arm into the control under a different name, and the numbers still look plausible;
3. versions and content hashes are locked and recorded per run.

Gate 2 has caught a real staging defect in this project[live], which is why it checks the provider-facing prompt and not an internal trace.

6 Reproduction: what the original design can and cannot express

Before measuring a disagreement it is worth establishing that there is no disagreement about the original's own numbers.

6.1 The published advantage survives cache-aware accounting

We recompute the original's headline table under our cost model: their reported token counts, our measured per-token tariffs, and our measured cache-hit rates for the two archetypal runtimes. The result narrows their advantage without reversing it[live]:

Horizon T	token ratio	cost ratio under caching
10	1.6×	**0.97**×
25	2.9×	1.75×
50	5.7×	3.42×
100	19.0×	11.45×
200	21.3×	12.82×

Two readings. Caching roughly halves the advantage but does not invert it: within the original design the method remains cost-positive, and we say so. And at T=10 the ratio has already crossed 1.0: cost parity disappears at the shortest horizon of their own table, unremarked because their metric cannot see it. This is A2 falsified from published data, with no new runs.

This estimate transfers our tariffs onto their tokens and is labelled as such: the order of magnitude is robust, the exact multipliers are not measurements of their spend.

6.2 The property that hides re-acquisition

In the original generator, horizon T is a count of environment events, and every runtime answers each event with exactly one model call[orig]. The number of model calls is therefore fixed by construction and identical across arms.

This single property determines what the evaluation can observe. The dominant cost of discarding history is re-acquiring what was discarded, and re-acquisition is spent in *additional agent-initiated steps*. In a design where the step count is an input, that cost has nowhere to appear. An agent that has forgotten a fact and must re-derive it still gets exactly T calls; the forgetting is invisible in the ledger the paper keeps.

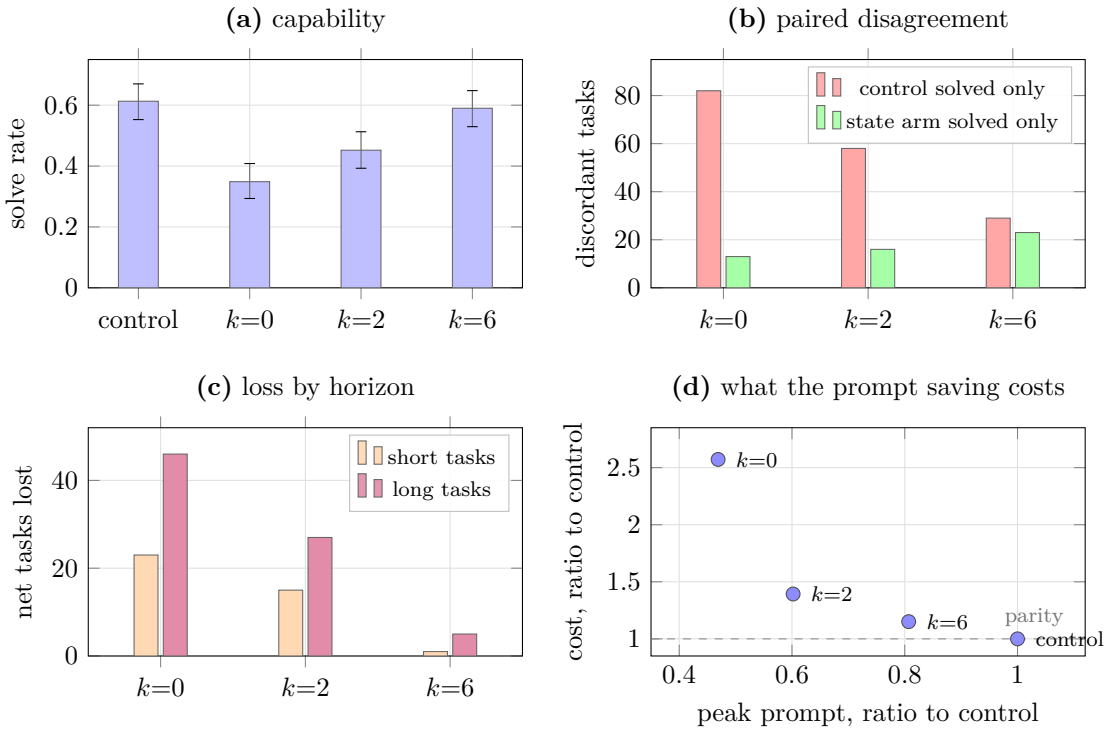


Figure 2: Characterising structured state memory on 87 coding tasks, 1044 runs. **(a)** Solve rate with Wilson intervals; the control solves 0.613 against 0.349 for the published configuration. **(b)** Paired disagreement against the same-trial control – only discordant tasks carry information, and they are overwhelmingly one-sided at k=0. **(c)** Net tasks lost, split at the control's median horizon: the deficit *widens* on long tasks, inverting the argument for the method. **(d)** At equal outcome the published configuration halves the peak prompt and multiplies cost by 2.57. Restoring six exchanges (k=6) removes the capability deficit and cuts the cost premium from 2.57× to 1.15×, but does not fall below the control: at equal outcome it remains the more expensive arm ($p = 0.04$).

In an agentic setting the step count is an outcome. On our corpus the state-only arm takes 2.22× the control's turns on tasks *both* arms solved[live], and that ratio is measured only where the method succeeded, so it is a lower bound on the overhead, not an average over its failures.

6.3 What this does and does not impugn

The original's mechanics reproduce, its token claim reproduces, and its sign survives caching *inside its own design.* What does not transfer is the inference from that design to agentic execution. Both are legitimate evaluations; they answer different questions, and only one of them is the question a practitioner asks before putting a memory architecture into a coding agent.

One fidelity caveat, stated here rather than buried: our state block is thicker per step than the original's reported figure[live], so our absolute cost magnitudes are upper bounds. A thinner block would strengthen the prompt-size reduction and weaken the cost inversion; both move against our headline, which is why we report both. [**NOTE:** the paper-fidelity arm that isolates this is built and pre-registered; if it runs before submission its numbers replace this paragraph]

7 Results I: capability

Figure 2 summarises this section and the next two; the tables below give the same quantities exactly.

Table 1: Solve rate by condition. 1044 runs, 87 tasks × 3 trials. Intervals are Wilson score.

Condition	Solved / runs	Rate	95% CI
Control (full transcript)	160 / 261	0.613	[0.553, 0.670]
State-only (k=0)	91 / 261	0.349	[0.293, 0.408]
State +2	118 / 261	0.452	[0.393, 0.513]
State +6	154 / 261	0.590	[0.529, 0.648]

Table 2: Paired capability against the same-trial control, exact McNemar. Only discordant cells carry information.

Condition	Pairs	Control only	Arm only	Both	p
State-only (k=0)	261	82	13	78	2.1×10^{-13}
State +2	261	58	16	102	9.7×10^{-7}
State +6	261	29	23	131	0.49

7.1 Solve rate

Table 1 gives the unpaired picture. The published configuration solves 0.349 of runs against the control's 0.613, and the intervals do not overlap. Restoring history recovers the rate monotonically: 0.452 at two exchanges, 0.590 at six.

7.2 Paired comparison

Unpaired rates understate the design: every condition ran every task, so the informative quantity is the set of tasks on which two arms disagree. Table 2 pairs each arm against the control of the same trial.

The published configuration loses 82 tasks and wins 13 ($p = 2.1 \times 10^{-13}$). The effect is large, one-sided, and far past the significance threshold. At two retained exchanges the deficit is still clear (58 vs. 16, $p = 9.7 \times 10^{-7}$). At six it is gone: 29 vs. 23 is indistinguishable from noise ($p = 0.49$).

That last row is the most informative in the paper. Six verbatim exchanges are a small addition to the prompt, yet they restore capability to parity. The damage therefore is not caused by summarising state, and it is not caused by structuring it: it is caused by *discarding the recent transcript*. What the state block costs in capability is repaid by the history it replaces, not by the state block itself.

7.3 Horizon: the loss grows where the benefit was promised

Tasks are split at the control's median turn count (11 turns), giving 45 short and 42 long tasks. Horizon is taken from the *control* arm: using the treatment's own turn count would be circular, since an arm that re-fetches takes more turns by construction.

For the published configuration the deficit is 28 against 5 on short tasks ($p = 6.6 \times 10^{-5}$) and widens to 54 against 8 on long ones ($p = 1.7 \times 10^{-9}$).

This inverts the argument for the method. Structured state is motivated by long horizons, where an unbounded transcript is supposed to become unmanageable. On this corpus the long horizon is where discarding the transcript costs most, so the deficit accumulates over a task in the same direction the benefit was expected to.

At k=6 neither stratum shows a difference, so the horizon effect is a property of how much history was removed, not of the state block.

Table 3: Paired capability by horizon stratum, split at the control's median of 11 turns.

Condition	Horizon	Control only	Arm only	p
State-only (k=0)	short	28	5	6.6×10^{-5}
State-only (k=0)	long	54	8	1.7×10^{-9}
State +2	short	23	8	0.01
State +2	long	35	8	4.2×10^{-5}
State +6	short	11	10	1.00
State +6	long	18	13	0.47

Table 4: Stop reasons by condition. Budget stops are concentrated in the arm that takes the most turns, which is why they are excluded from cost aggregation.

Condition	output_ceiling	solved	turn_budget	unsolved
Control (full transcript)	62	160	0	39
State-only (k=0)	117	91	34	19
State +2	95	118	18	30
State +6	67	154	0	40

7.4 How runs ended

A capability comparison is only honest if a harness limit is not being counted as an inability. Table 4 separates the four terminal states.

Turn-budget exhaustion appears in the state-bearing arms and never in the control or at k=6[live]. This is itself evidence for the mechanism of §8: the arm that must re-acquire its context is the arm that runs out of turns. It is also the pre-declared artefact trap of §4, and it is why every cost claim in §9 is computed only on pairs where both arms actually solved the task.

8 Results II: mechanism

Section 7 establishes that discarding recent history costs capability. This section establishes *how* the cost is paid, from the tool calls of the same runs.

8.1 The agent buys back what the prompt discarded

A state-only run does not fail by forgetting its plan. It fails by re-deriving what it already observed. Classifying every shell command of 472 runs with full traces by purpose, and counting commands that repeat an earlier identical invocation[live]:

Condition	Commands / run	Inspect share	**Repeated inspect**
CONTROL (full history)	95	0.23	**0.20**
STATE-ONLY, solved	163	0.52	0.45
STATE-ONLY, failed	278	0.67	**0.58**

A failing state-only run spends 58 % of its commands re-running an inspection it has already run, against 20 % for the control. On a separate matrix the same effect appears as a pointer-follow rate of 5.19 per run against the control's 0.42, roughly twelve times as often[live].

The state block bounds the *prompt* without bounding the *information the task requires*, and the shortfall is settled through the tool loop, where no prompt-size metric looks. That is the thesis of this paper stated as a measurement rather than an argument.

8.2 Read by hand

One trajectory, inspected directly rather than through an aggregate[live]: 72 tool calls, of which `input.tex` is read 22 times across two spellings of its path, a second file 19 times, and the document is recompiled five times, returning the same seven warnings at the same line numbers on every invocation. Turns 22–33 are almost entirely alternating re-reads of two files whose bytes never changed.

Aggregates can be produced by a broken metric; this one was read first and the metric written afterwards to reproduce it mechanically.

8.3 What the state actually contained

Examining the final state of 251 state-only runs by outcome[live]:

Marker present in the final state	Solved	Failed
Characters (median)	1218	963
File paths	9	5
Exact version strings	1	**0**
Error or exit-code text	0	**0**
Quoted commands	1	**0**

A failing run's state carries zero exact versions, zero error text and zero commands, precisely the facts it then re-derives by running the same command again. The schema has nowhere to put them: its fields are prose buckets (`goal`, `facts_established`, `work_done`, `work_remaining`, `next_step`, `current_blocker`).

The association runs the expected way[live]: runs whose state records at least eight file paths solve at 0.50 against 0.24 for those that record fewer; runs recording an exact version solve at 0.47 against 0.32; runs recording an error or exit code at 0.50 against 0.33. This is correlational (a run that got further had more to record), but it is the only direction the data points, and it motivates the composition study of §10.

8.4 Why this cannot appear in the original evaluation

Re-acquisition is spent in extra agent-initiated steps. The original's generator fixes the step count per episode (§6), so the mechanism measured in this section has no channel through which to register. This is the concrete sense in which the synthetic result is not wrong but inapplicable: the benchmark removes the variable that carries the cost.

9 Results III: economics

9.1 Cost at equal outcome

Comparing cost across arms that solve different numbers of tasks is meaningless: a failing run stops early and therefore costs less. Every ratio in Table 5 is computed only on pairs where *both* arms solved the same task in the same trial, and each ratio is a per-pair median, so one expensive task cannot set the number.

At equal outcome the published configuration carries 0.47× the control's peak prompt and costs 2.57× as much ($p = 1.4 \times 10^{-12}$). The prompt is half the size and the bill is two and a half times larger.

Table 5: Resource ratios against the same-trial control, on pairs where both arms solved the task. Per-pair medians; p from an exact sign test on the cost ratios.

Condition	Pairs	Cost	Peak prompt	Output	Cached	p
State-only (k=0)	78	2.57×	0.47×	4.55×	0.89×	1.4×10^{-12}
State +2	102	1.39×	0.60×	2.08×	0.40×	4.7×10^{-4}
State +6	131	1.15×	0.81×	1.36×	0.21×	0.04

Table 6: Series totals. Reported for scale only: per-condition sums are not a fair comparison, because a failing run stops early.

Condition	Cost	Turns	Input	Cached	Output
Control (full transcript)	$17.85	6,504	18.7M	352.2M	9.0M
State-only (k=0)	$14.83	16,072	27.3M	25.9M	19.9M
State +2	$12.88	10,250	27.6M	15.9M	16.5M
State +6	$11.83	4,725	38.9M	6.2M	11.6M

9.2 Where the money goes

The same table carries the mechanism. Output tokens are 4.55× the control's: the state block is regenerated every turn, and generated text is billed at the most expensive tariff. Cached reads fall away as the retained history shrinks, and the series totals show the same effect from the other side: the control is served 352.2M cached tokens across the series against 6.2M for STATE+6^{live}.

An append-only transcript is almost entirely cache-served, because every call shares the previous call's prefix. A state block rewritten each turn diverges from the previous prefix at the point where the block begins, so the discount that makes the control cheap does not apply. Neither quantity is visible in a cumulative token count, which is the concrete content of the methodological claim: token reduction and cost reduction are different measurements, and on this corpus they have opposite signs.

9.3 Solved tasks per unit spend, at series level

A runtime that halves tokens and fails half the tasks is not an improvement. The efficiency view (Table 7) divides outcomes by resources.

Two things follow. Per token the state arms do look efficient (1.24 against the control's 0.42 solved per million); this is the original's metric, and it is the reading that a token-count evaluation produces. Per dollar the ranking changes: STATE-ONLY is the worst of all four arms at 6.1 solved per dollar, below the control's 9.0.

By that series-level measure the best arm is STATE+6 at 13.0 solved per dollar against 9.0. That comparison must be read with the caveat of §9.1: a failing run stops early and costs little, so series totals flatter whichever arm fails more, and they are reported here for scale rather than as a verdict.

The matched-outcome comparison is the one that decides. On it STATE+6 is *not* cheaper than the control: 1.15× over 131 tasks that both arms solved, more expensive in 78 of them against 53 ($p = 0.04$)^{live}. The honest statement of the economics is therefore narrower than a token count suggests in either direction. Restoring a short tail removes the capability deficit (§7) and cuts the cost premium from 2.57× to 1.15×, but on this corpus no configuration of structured state was cheaper than simply keeping the transcript at equal outcome.

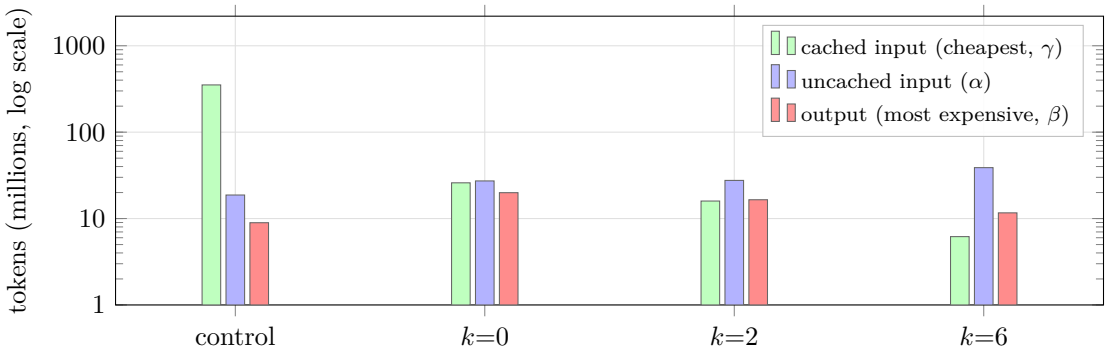


Figure 3: Token consumption by billing class across the series, the decomposition Eq. (8) prices. Bars are grouped, not stacked, and the axis is logarithmic because the classes differ by two orders of magnitude. The control consumes by far the most tokens in total, but 352.2M of them are cached reads billed at the cheapest rate: an append-only prompt is almost entirely served from the prefix cache. The state arms consume far fewer tokens and pay more for them, because a state block rewritten each turn forfeits that cache (down to 6.2M) and because maintaining the state moves tokens into the output class. This is why a token-count comparison and a cost comparison disagree.

Table 7: Efficiency. Solved tasks per million tokens and per dollar across the whole series.

Condition	Solved	Tokens	Cost	Solved/Mtok	Solved/$
Control (full transcript)	160	379.9M	$17.85	0.42	9.0
State-only (k=0)	91	73.1M	$14.83	1.24	6.1
State +2	118	60.1M	$12.88	1.96	9.2
State +6	154	56.7M	$11.83	2.72	13.0

9.4 H2 refuted as pre-registered

The pre-registration predicted that the cost ratio would fall with horizon and cross 1.0; that is, that the overhead would amortise on long tasks. It does not. The Spearman correlation between control horizon and per-task cost ratio is +0.16 over 35 tasks ($p = 0.36$), a correlation indistinguishable from zero and, if anything, of the wrong sign.

The test covers 35 of the 87 tasks by the definition of the quantity, not by any exclusion rule: a per-task cost ratio exists only where the control and STATE-ONLY both solved the task in at least one trial, and STATE-ONLY solves 0.349 of runs. The restriction follows from the capability result of §7, and it cuts against us: the tasks where STATE-ONLY failed outright are the long ones on which the amortisation was predicted.

No crossover exists within the observed horizon range. We report this as a refutation of our own hypothesis rather than dropping it, and it converges with §7: not only does the cost fail to amortise, the capability deficit widens with horizon.

10 State composition: who writes the state

Sections 7–9 establish that the damage comes from discarded history and that the agent pays for it by re-acquiring information through its tools. That diagnosis implies a repair: if the state already carried the facts the agent keeps re-deriving, the re-acquisition would not be necessary.

This section varies one thing: how much of the state the *runtime* writes without asking the model. It is a different axis from the tail, and it is reported separately because it answers a different question. The tail axis asks how much history to keep. This axis asks, for a fixed amount of history (none), who should produce the state that replaces it.

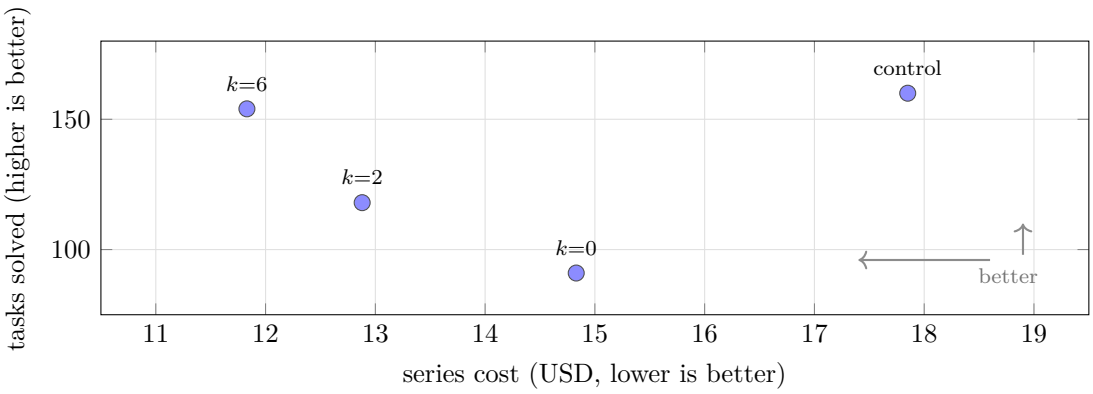


Figure 4: The frontier that matters in practice: tasks solved against money spent, whole series. The published configuration (k=0) is dominated: it spends less than the control but solves far fewer tasks, and both k=2 and k=6 beat it on both axes simultaneously. Read this figure as a series-level summary only. Because a failing run stops early and therefore costs little, series totals flatter the arms that fail more, which is why the matched-outcome comparison of Table 5 and not this figure carries the paper's cost claim. On that fair comparison k=6 is *not* cheaper than the control (1.15×, $p = 0.04$). What the figure does show is that neither extreme of the tail axis is the right answer.

10.1 The factor

In SKILL.STATE the state block is generated text: the model emits $\Delta\Sigma_t$ in the same completion as its action (Algorithm 2). Two consequences follow from §9, and both are unfavourable. Maintaining the state is billed at the output tariff β, the most expensive class in Eq. (8). And the model is free to overwrite a fact it recorded earlier; the original's own error taxonomy attributes 68 % of open-weight failures to exactly that[orig].

The alternative is to let the runtime record what already happened. A tool result is an observed fact, not a claim: the runtime saw which files were touched, which command ran, what it printed, and whether the tests passed. Writing those into the state requires no generation and cannot be overwritten by the model.

How much to record is the factor. The ladder adds one component per rung:

Rung	Adds	What the runtime writes
v1	files	paths the agent has touched
v2	+ content	per-file and global modification records
v3	+ commands	normalised command records with their output
v4	+ diagnostics	compiler and linter output, structured
v5	+ environment	versions, paths, interpreter state
v6	+ tests	test invocations and their results
v7	+ failures	records of what was tried and did not work

The tail is frozen at k=0 and the schema at `L2-semantic` in every cell, so no rung borrows an advantage from the factors of §7. Neighbouring rungs differ by exactly one component, and that invariant is asserted over the flags the conditions actually emit before any cell is run, rather than over the intent behind them[code].

The control runs inside this series. Reusing E1's control was tried and the data refused it: the same arm on the same 87 tasks scored 0.793 in E1 and 0.552 here, disagreeing on 31 of them[live]. The stand drifts between series, so a borrowed control measures the drift as well as the condition. Against the borrowed control the first rung looked like a 46–2 deficit; against the in-series control it is 25–2. One control per series.

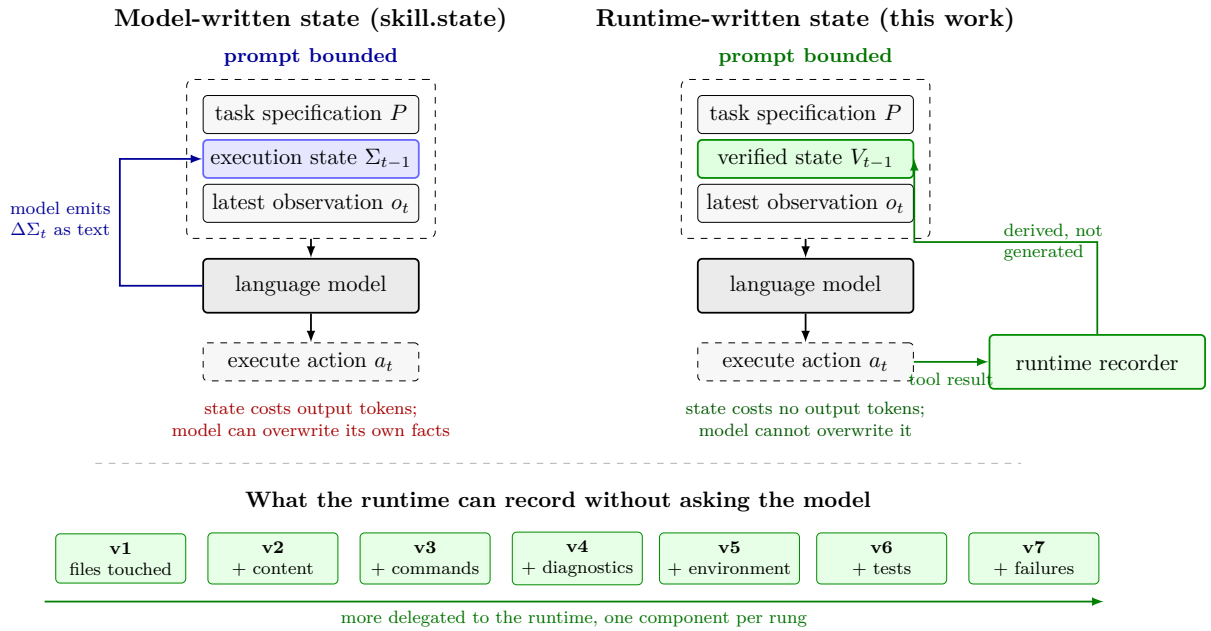


Figure 5: Two ways to bound the prompt with a state block, differing in who writes it. **Left:** the model emits the state update as generated text, so maintaining it is billed at the output tariff and the model is free to overwrite a fact it previously recorded, which the original attributes 68 % of its open-weight failures to[orig]. **Right:** the runtime derives the same record from tool results that have already occurred. The block is a fact about what happened rather than a claim the model makes about it, costs no generated tokens, and cannot be overwritten by the model. The bar below is the factor varied in §10: each rung hands one more component to the runtime, with tail and schema frozen, so a gap between neighbours names a single delegated component.

10.2 Capability improves as more is delegated

Table 8 gives the ladder. The first rung, a state holding only the list of files touched, solves 0.287 against the control's 0.552. Adding command records lifts it to 0.402, tests to 0.425. The last rung does not continue the rise: adding failure records gives 0.414, one task fewer than v6, which on 5 against 4 discordant tasks is indistinguishable from no change. The ladder is therefore not monotone at its top, and we do not claim it is.

The paired steps are in Table 9. Only the first is significant on its own: moving from files to commands wins 12 tasks and loses 2 ($p = 0.01$). The later single steps are individually indistinguishable from noise at this sample size (9 vs. 11, $p = 0.82$; and 5 vs. 4, $p = 1.00$). The end-to-end step is significant: v1 to v7 wins 14 and loses 3 ($p = 0.01$).

We therefore claim the trend and not the individual rungs. What the data supports is that a state carrying more runtime-derived facts solves more tasks than a state carrying fewer, with most of the gain arriving early, at the point where commands and their output enter the record. Locating the remaining components against each other needs the trials still running.

10.3 Cost falls at the same time

The result that matters is in the last three columns of Table 8, because capability and cost move together rather than against each other.

At v1 the arm costs 2.31× the control at equal outcome. This is the regime of §9: a state that records almost nothing forces the agent to re-acquire almost everything, and it takes 1.62× the control's turns to do it. By v6 the same comparison is 0.92×, below the control, and turns have fallen to 0.71×. The arm that delegates the most is both the most capable on this axis and the cheapest.

Table 8: The composition ladder. Cost, turns and output are medians over tasks that both the rung and the control solved; ratios below 1 favour the rung.

Rung	Adds	Solved	Rate	Cost ratio	Turns	Output
CONTROL	full transcript	48	0.552	1.00×	1.00×	1.00×
v1	files	25	0.287	2.31×	1.62×	3.96×
v3	+ commands	35	0.402	1.20×	0.79×	1.64×
v6	+ tests	37	0.425	0.92×	0.71×	1.84×
v7	+ failures	36	0.414	0.94×	0.73×	1.73×

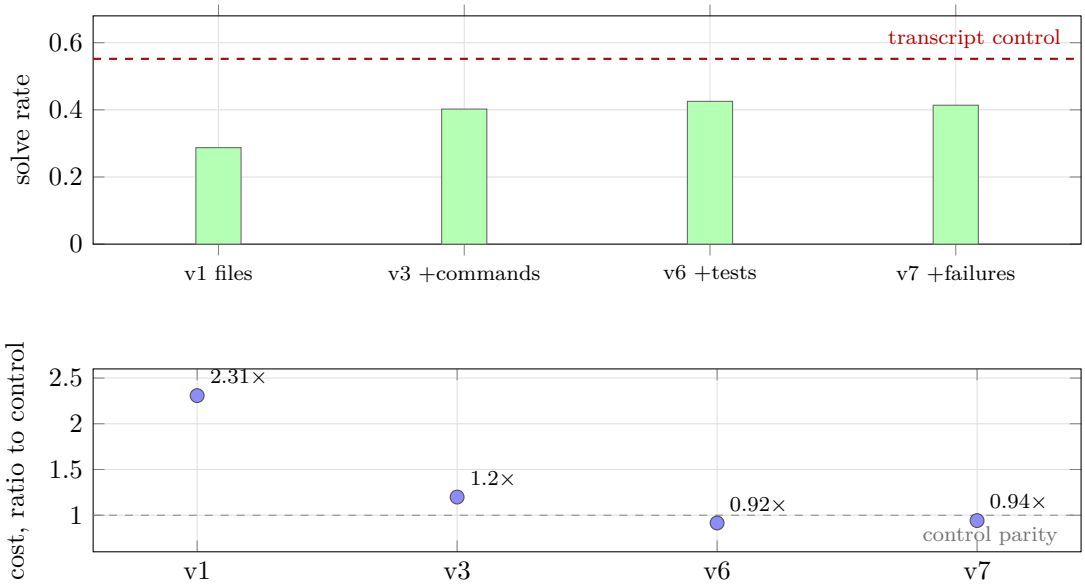


Figure 6: Delegating more of the state to the runtime moves capability and cost in the same direction, which is the opposite of the usual trade. **Top:** solve rate by rung against the in-series transcript control (dashed). The first rung recovers 0.287 of tasks; handing over commands, tests and failure records reaches 0.425, closing roughly half the distance to the control without restoring any transcript. **Bottom:** median cost on tasks both the rung and the control solved. The same additions that raise capability also take cost from 2.31× the control down to 0.92×, below parity. A state the runtime derives is cheaper than a state the model writes, because it is never generated at the output tariff.

The mechanism is visible in the output column. At v1 the arm generates 3.96× the control's output tokens; by v6 it is 1.84×. Facts the runtime writes are not paid for at the output tariff, so moving a component from the model to the runtime removes generated tokens at the same time as it removes the re-acquisition those tokens were failing to prevent.

Per dollar across the whole series the ordering is the same: 12.9 solved tasks per dollar at v1 against 20.9 at v6, with the control at 22.4.

10.4 What this does not show

The ladder does not reach the control. Every rung remains behind it on paired capability (Table 10), and the best is still -11 tasks net against a transcript agent. Delegating more to the runtime is a strict improvement *over a model-written state*; on this corpus it is not yet a replacement for keeping the recent history.

Combined with §7, where six verbatim exchanges alone close the gap entirely, the practical reading is that the two mechanisms are complementary rather than competing: the runtime-written state is the cheap way to carry durable facts, and a short tail is the cheap way to carry

Table 9: Paired steps along the ladder, same task and trial.

Step	Lower solves only	Higher solves only	p
v1 → v3	2	12	0.01
v3 → v6	9	11	0.82
v6 → v7	5	4	1.00
v1 → v7	3	14	0.01

Table 10: Each rung paired against the in-series transcript control.

Rung	Control only	Rung only	p
v1	25	2	5.6×10^{-6}
v3	18	5	0.01
v6	17	6	0.03
v7	16	4	0.01

recent context. Neither substitutes for the other here.

Status of these numbers. 435 runs, 87 tasks, one trial. E1 carries three trials; this axis carries one, and this project has already had an effect that was significant at two trials and died at the third (§12). The direction is consistent across every rung and the end-to-end comparison is significant, but the individual rung-to-rung steps should be read as provisional until the remaining trials land. No claim in this section rests on the plugin's own instrumentation counters, which are identically zero across the series and are therefore treated as dead rather than as evidence of absence[live].

11 Threats to validity

One model family. Every run uses `zai/glm-5.3-flash`. A paired pilot on a stronger model from the same provider solved no task the small model could not, at 22.7× the median cost per cell[live], which is why the series was not moved. That pilot is 7–8 tasks and settles a budget question, not a transfer question. We claim nothing about models from other families, and in particular nothing about whether a model with stronger instruction-following would maintain state well enough to avoid the failure mode of §8.

One corpus, and its horizon range. Terminal-Bench 2.0 tasks utilise a median of roughly 5 % of the context window under the control[live]. The window is therefore rarely the binding constraint, which is the regime in which a memory mechanism should be least necessary. A pilot on the longer Terminal-Bench 4.0 shows a median peak of 66 % of the window[live], and there the same mechanism appears an order of magnitude stronger (median repetition 3.26 against the control's 1.02). That pilot is one trial on six tasks and is not a result. The honest statement of scope is that our finding holds where the window is not saturated, and that the direction of the effect in the saturated regime is measured but not established.

Implementation fidelity. Our state block is thicker per step than the original reports[live], so absolute cost magnitudes are upper bounds (§6). Both corrections move against our headline.

Tariff transfer in the reproduction. Section 6 applies our measured prices to the original's published token counts. The order of magnitude is robust; the exact multipliers are an estimate

and are labelled as one.

Contamination not detectable in principle. The quarantine rule (§12) keys on errored turns. A provider degradation below the error-visibility floor would not be caught. The bound on the damage is that the cost results do not depend on verdicts at all, and that the capability result survived removal of every cell the rule did catch.

Two unrunnable tasks. Two of 89 tasks fail during container setup in every condition^{live} and are absent from all arms equally. This removes two tasks from the corpus; it does not favour a condition.

The composition study carries one trial. Section 10 reports 435 runs at a single trial, against three for E1. Given this project's own history (an effect significant at two trials that died at the third (§12)), the individual rung-to-rung steps there are provisional. The end-to-end comparison and the direction of the cost trend are consistent across every rung, which is why they are stated at all, but no claim in this paper's abstract or conclusion rests on them.

No tail-only arm. The four E1 conditions are a control with no state block and three state-bearing arms at different tails. There is no arm carrying the last k exchanges and *no* state block. Without it, §7 can show that restoring history removes the deficit, but cannot separate how much of the remaining behaviour at k=6 comes from the state block rather than from the history beside it. Table 6 contains a hint that this matters: STATE+6 completes the series in fewer total turns than the control, which a purely additive account of the two components does not obviously predict. We flag it as unexplained rather than interpret it, and the arm belongs in the next series.

12 Measurement pitfalls, and two retractions

Agent-memory evaluations have a specific failure mode: the numbers are real, the script is correct, and the label on the column is wrong. Each item below cost this project a series before it became a rule, and each is reported because the rule transfers even where our results do not.

12.1 Five pitfalls

1. **A model's self-report is not a verdict.** Only the task's own verifier decides. Every alternative we tried produced rows that looked like data and were not.
2. **A treatment arm can silently become the control.** A plugin that fails to load leaves an arm running as the baseline under a different name, and its numbers remain plausible. The gate asserts the realised context composition against the provider-facing prompt, not against the plugin's own log^{code}.
3. **A detector can match configuration instead of events.** A timeout detector in this project matched a static retry-policy line printed in every log, reporting 54 timeouts where there were none^{live}. The defect surfaced only through an impossible downstream number: a solve rate above 100 %. Its unit test had encoded the same mistake, so the suite was green because code and test were wrong identically. Rule adopted: a detector is validated against a run whose ground truth is known before it is allowed near the corpus.
4. **Provider saturation reads as method failure.** Run at a concurrency above the provider's measured ceiling, 68 % of requests were rate-limited^{live}. A starved run still receives an honest zero from its verifier, and that zero enters the table as "the method

failed". The contamination is one-directional by construction: the control had already completed in clean conditions, so only the treatment arms lost turns. The quarantine rule was fixed before the counts were seen: clean means zero errored turns, nominal at most two and under 5 %, everything else excluded.

5. **Underpowered binary endpoints.** Before this series we computed that detecting a solve-rate difference required roughly 72 tasks at the measured discordance rate, and said so rather than reporting a null as an equivalence. The present series clears that requirement, which is why §7 makes a capability claim at all.

12.2 Two retractions

Both are reported because a retraction is a result about measurement.

"Time-limit censoring explains the capability gap": withdrawn; the censoring never existed. This was the successor claim to the detector defect in item 3. Once the detector was corrected, the true count of time-censored runs in the corpus was zero. The follow-up observation that raising the limit recovered four of seven failed tasks was then tested against the measured base rate of verdict recovery between unchanged repeat runs (5/11); the probability of seeing four or more recoveries from that base rate alone is 0.40^{live}. Unremarkable, and withdrawn.

"State-only is less stable": withdrawn at the third trial. After two trials the state-only arm showed higher trial-to-trial dispersion on 10 of 12 tasks ($p = 0.039$). The third trial moved it to 8 of 12 ($p = 0.388$). The claim died exactly as a two-trial p just under 0.05 should be expected to. Had the series stopped at two trials, a false effect would have entered this paper.

What the retractions did not touch. Every cost, token and cache measurement in this paper is independent of verdicts, and the capability results of §7 come from a design that postdates all of the above and clears the power requirement that motivated it.

12.3 An interim contamination case, kept as a warning

An early schema analysis in this project produced $p = 7.9 \times 10^{-8}$ in favour of a strong effect. Re-run on the quarantined-clean corpus with the same script, the same comparison gives $p = 9.2 \times 10^{-2\mathsf{live}}$. The contaminated cells were concentrated in the arms being compared. Nothing about the arithmetic was wrong; the column heading was.

13 Related work

Structured state memory. SKILL.STATE [Badhe et al., 2026] is the system under test and is described in §2. The family it belongs to (replacing history with a model-maintained structured representation) shares the assumption we falsify: that the representation is a sufficient statistic for the remaining work.

Deterministic execution state. LEDGER [Wang et al., 2026] maintains an objective record derived from completed interactions and adds it to an otherwise unmodified agent that keeps its history. Our composition study (§10) varies the boundary between the two designs. The difference in setting matters and we state it plainly: LEDGER *adds* a deterministic layer to an agent with full history, whereas we ask what a deterministic layer must contain when the history has been *removed*.

Context management by compression. Summarisation runtimes, sliding windows and learned prompt compression [Jiang et al., 2023] attack the same growth problem by shrinking the transcript rather than replacing it. The original's budget-matched controls report that at equal prompt size these score well below structured state^{orig}, which is evidence that structure rather than brevity carries its gains, a conclusion our results do not contradict. Our finding is about what is lost when the transcript is discarded entirely, not about how well it can be compressed.

Agentic reasoning loops. The transcript baseline is ReAct-style [Yao et al., 2023]: interleaved reasoning and acting with an append-only history. Its cost profile under prefix caching is what makes the comparison in §9 non-obvious, since the growing prompt is largely cache-served.

Benchmarks with programmatic verifiers. We use Terminal-Bench, whose tasks carry their own verifiers, for the reason given in §5: a verdict must not depend on a model's judgement of its own output. InterCode CTF [Yang et al., 2023] and τ-bench [Yao et al., 2024] are used by the original and share this property.

Evaluation methodology. Our methodological contribution (§12) applies standard reproducibility practice (external verdicts, pre-registered power, written retractions) to a genre where, in our experience, the dominant error is not statistical but definitional: a column that measures a property of the harness while being labelled a property of the method.

14 Conclusion

Structured state memory is motivated by a correct observation: an unbounded transcript is a poor working memory, and its cost grows with everything that came before. The published evaluation of SKILL.STATE shows that a model-maintained state block bounds prompt growth at near-constant size while preserving quality, and on its own benchmark that result reproduces, including under cache-aware accounting.

On real coding tasks it does not hold. Across 1044 runs of a pre-registered design, the method as published solves 0.349 of tasks against 0.613 for a transcript control, losing 82 paired tasks and winning 13 ($p = 2.1 \times 10^{-13}$), with the deficit *widening* on long horizons where the mechanism is supposed to pay for itself. Restoring six verbatim exchanges beside the state block removes the difference entirely, which places the damage in the discarded history rather than in the state abstraction.

The reason the two settings disagree is structural rather than empirical. The synthetic benchmark fixes the number of model calls per episode, so an agent that has discarded something it later needs cannot spend anything to recover it. Real agents can, and do: in failing state-only runs 58 % of all commands are repeats of an inspection already performed, against 20 % for the control. This is the paper's thesis in one sentence: *bounding the prompt does not bound the information the task requires; the difference is paid through the tool loop, where no prompt-size metric looks.*

The economics follow from the same mechanism and invert the usual reading. At equal outcome the state-only arm carries half the control's peak prompt and costs 2.57× as much, because state maintenance is billed at the output tariff and a block rewritten each turn forfeits the prefix cache that makes an append-only transcript cheap. Measured per token the state arms look efficient; measured per solved task per dollar over the whole series the published configuration is the worst of the four. On the matched-outcome comparison that this paper treats as the fair one, however, every state configuration including the best one costs more than the control (1.15× at k=6, $p = 0.04$).

The practical conclusion is therefore not that structured state should be abandoned, but that it should not be used to *replace* recent history. The configuration that wins on this corpus keeps both, and the open question we are measuring next is what a runtime-maintained state must contain for the remaining gap to close (§10).

The methodological conclusion may outlast the empirical one. In this genre the difference between a finding and an artefact is rarely the model, the method or the benchmark. It is the accounting: which currency the cost is measured in, which arm a harness limit penalises, and whether the column heading still describes what the column contains.

References

A. Badhe, A. Tiwari, and J. Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. v3, 02 Sep 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023.

Y. Wang et al. Ledger: Deterministic execution state for coding agents. *arXiv preprint arXiv:2608.00808*, 2026.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

A Reproducibility

The dataset. 1044 runs, one row per run, with per-row provenance: run identifier, start and finish timestamps, repository revision, plugin content hash, schema hash, model, trial, condition, realised k, verifier verdict, stop reason, turn and call counts, the three token counts separately, peak prompt, and derived cost. Every row was cross-checked field by field against the raw collector output with zero discrepancies[live].

Regenerating the paper's numbers. Every table, every figure coordinate and every inline figure in this paper is produced by one deterministic analysis from the released run data. No measured number is written into the manuscript source; the text references generated definitions, so a change in the data cannot leave a stale number in the prose.

The analysis additionally enforces an integrity gate, stated as Algorithm 3: it recomputes the published headline values from the raw rows and refuses to emit the document if any of them disagrees. The gate was verified by construction: corrupting a sample of verdicts in the input causes it to fail and to name every affected quantity[live].

The exact invocation is given in the repository's build instructions rather than here.

Algorithm 3 Build-time integrity gate

Require: canonical run table D; published headline values V

recompute every table and inline quantity from D
$M \leftarrow$ the recomputed headline values
for each $v \in V$ **do**
    if $M[v] \neq V[v]$ **then**
        report v with both values
        abort the build
    end if
end for
emit tables, figure data and macro definitions
typeset the document

Statistics. Exact McNemar and exact sign tests are computed from the binomial distribution in closed form; Wilson intervals are used for proportions; the horizon trend uses Spearman rank correlation with a permutation p-value under a fixed seed. The implementation depends only on the standard library, so it runs unchanged on the measurement host.

Exclusions, with counts. Attempts that returned no verifier verdict are recorded separately and excluded; the large majority were re-queued and later closed with a real verdict, and the residue is symmetric across arms. Runs measured under provider saturation are quarantined by the pre-declared rule of §12. Where an orchestrator restart ran one cell twice, the builder keeps the earliest launch, a rule fixed before any verdict was read so that a repeat cannot promote itself by winning^{code}.

Stack. Agent, harness and plugin versions are pinned and recorded per run; content hashes for the plugin and schema are written into every row, so a run can be attributed to the exact code that produced it.

Artefacts. The dataset, the raw collector output, the analysis and the manuscript source are released together. [**NOTE:** Zenodo DOI to be minted at submission; repository URL to be filled once the release is public]

B Reporting gaps in the original evaluation

Compiled during a full extraction of the original paper, each checkable against it directly^{orig}. These are recorded because several of them are the reason a reproduction must make its own choices, and a reader should know which choices were forced.

1. **Schemas.** A verbatim field list is given for one of the four evaluation domains. For two others the schema is inferable only from a single example trajectory; for the fourth none is given. No schema-design method is described anywhere, and there is no sensitivity analysis. A reproducer must therefore invent a schema, which is itself a source of divergence, and the motivation for treating schema design as a factor rather than a constant.
2. **Caching and money.** A full-text search returns zero occurrences of cache, billing or monetary terminology. All economic claims are in logical token counts. The paper is silent on the question, not brief.
3. **Variance and repetitions.** The noise-robustness and state-recovery tables report point values with no dispersion and no stated number of repetitions.

4. **Compression control underspecified.** The learned-compression baseline is reported without a compression ratio or variant. The truncation budget is stated in characters in one place and tokens in another.

5. **Statistical claim not shown.** Significance is asserted as a blanket paired t-test at $p < 0.01$ for horizons ≥ 50, with no per-comparison values and no correction for multiple comparisons.

6. **An unremarked null.** In one open-weight configuration at the longest reported horizon, the structured-state runtime and a stateful baseline score identically. The table contains it; the text does not discuss it.

7. **Step-count authority.** Horizon is a count of externally generated events, and no condition varies who controls the step count. This is the property analysed in §6, and it is the single most consequential difference between that evaluation and ours.